

SYSTEM DESIGN DESCRIPTION

1 INTRODUCTION

Provide a high-level overview of the system and include additional information that may be appropriate. The SDD is the system development blueprint that describes the system in detail. The details of the SDD are developed to the requirements and show traceability back to those requirements.

1.1 Background

Discuss the organization for which the system is being developed, including its major responsibilities. Provide additional information to give the reader some context for the system, for example the capabilities gap the new or revised system will overcome or the strategic goals the system is intended to fulfill.

1.2 References

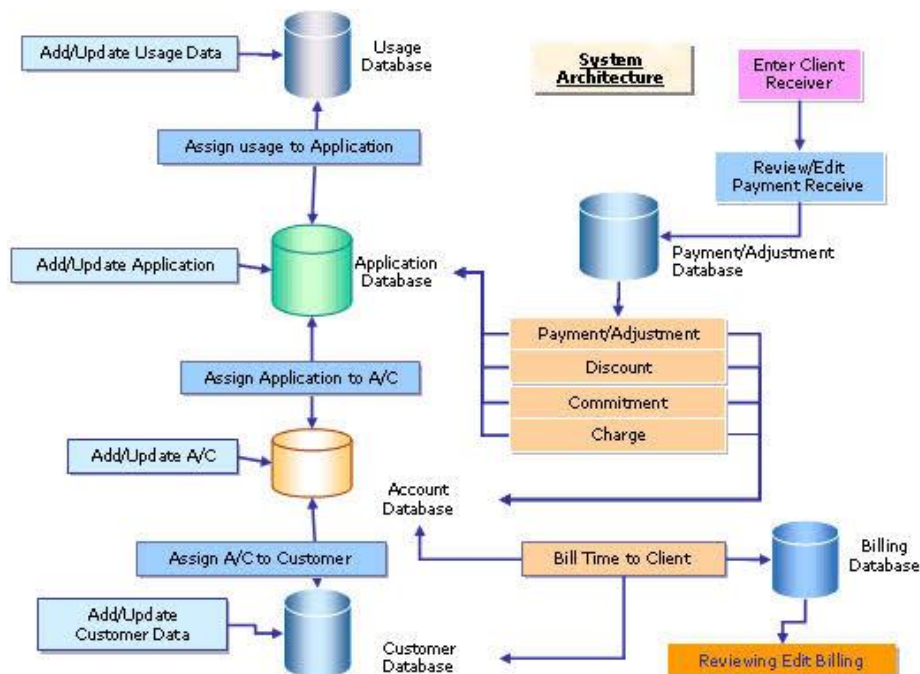
List key documents that support the SDD. Include meeting summaries, white paper analyses, standards, and related deliverables, as well as preceding documentation that provides information for its development. Keep it brief, details or a long list can go in an appendix. Pointing to documents that are frequently updated rather than including the information here (such as the risk register) will ensure this document remains current.

2 Functional Decomposition - SYSTEM DESCRIPTION

Describe the system in narrative form using non-technical terms.

2.1 System Software Architecture

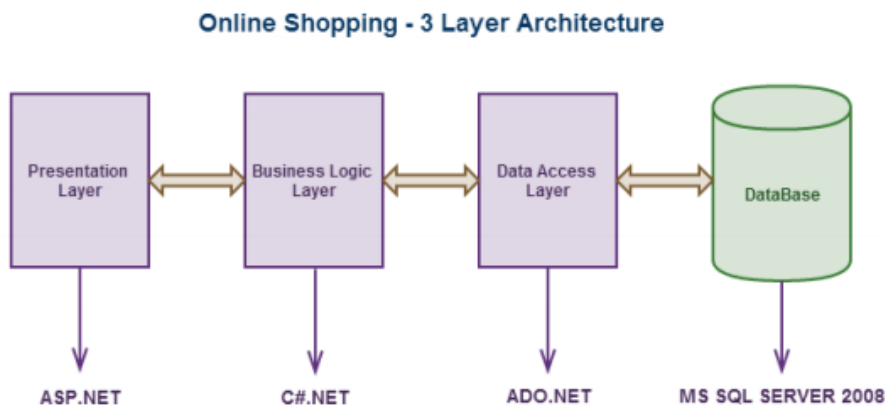
Provide an overview of system software architecture and organization and a high-level diagram illustrating the subsystems and show interfaces to external systems and relationships between the system and user organizations.



Architecture of billing software

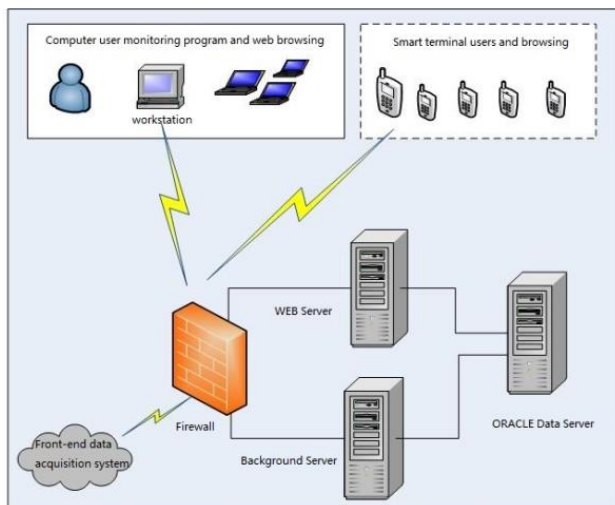
2.2 System Technical Architecture

Describe and diagram the technical platform(s) that will be used to build the. If different subsystems use different technical platforms, describe each platform and relate it to the appropriate subsystem.



2.3 System Hardware Architecture

Describe the overall system hardware/communications and its organization. Include a list of hardware components, a brief description of each item, and diagrams showing connectivity.



Provide a description of the equipment required to operate the system. Include descriptions of the equipment presently available, as well as a more detailed description of the characteristics and impact of new equipment necessary for system operation. If the equipment documentation is extensive, put the information in an appendix or referenced as a separate document. Add diagrams and information to describe each component and its functions. Follow industry-standard component specification practices. Include the following information in the detailed equipment component designs:

- A. Memory and/or storage space requirements
- B. Processor requirements (speed and functionality)
- C. Graphical representation depicting the number of hardware items (for example, monitors, printers, servers, input/output devices), and the relative positioning of the components to each other

2.4 External Interfaces

Describe interfaces with other application software units, including those of other operational capabilities internal and external. For each interface, specify the following information:

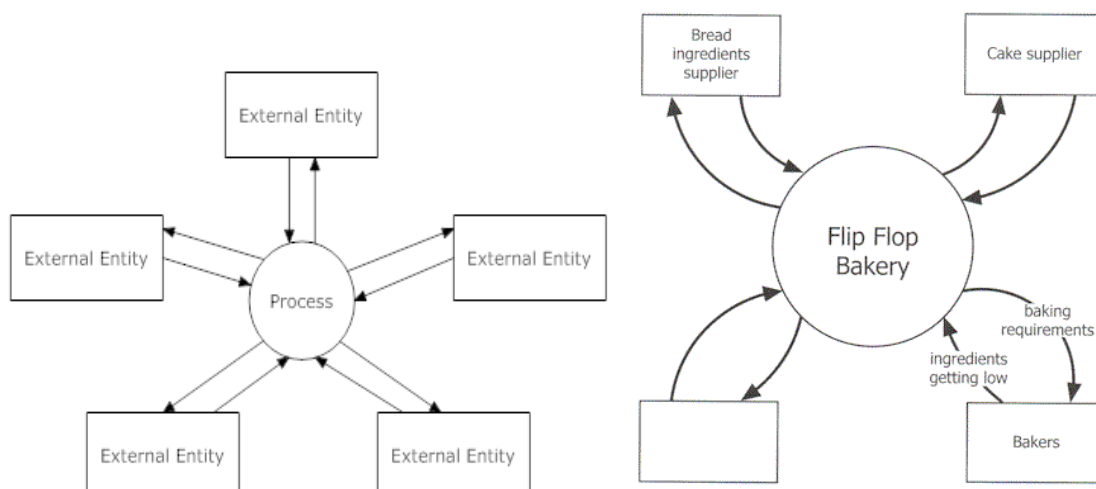
- A. Name of application
- B. Owner of application, if external
- C. Details of interface
- D. Type of interface, such as interfaces to other software units
- E. Description of operational implications of data transfer, including security considerations
- F. Data transfer requirements to and from the software unit, including data content, format, and sequence
- G. Formats of data for both the sending and receiving systems, including the data item names, codes, or abbreviations that are to be interchanged
- H. Interface procedures
- I. Interface equipment
- J. Data conversion requirements

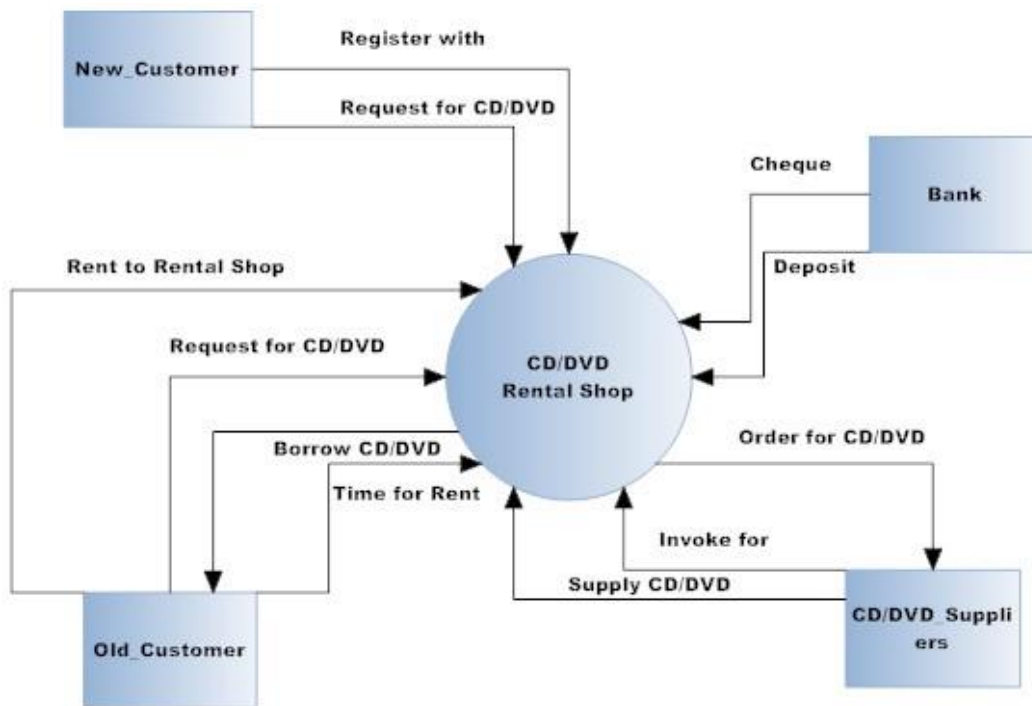
3 Description of Programs – Subsystem Specification

List and describe subsystems to establish a reference within the document. For web based systems, different web pages can be categorized as a subsystem. Provide a diagram, such as a hierarchical structure chart, that depicts the flow and mapping of the entire system. Include graphics of relationships of user organizations to major system components. To aid in identifying the set of requirements represented by those diagrams, it is recommended that the requirement number(s) be provided.

3.1 Context Flow Diagram (CFD)

A "context level" DFD can be used to show the interaction between a system and outside entities (users, other hardware, other software); it can also show the internal data flows within a system. This version is also called a context diagram. It often shows the information system as a single circular shape (bubble) with no details of its inner workings: what it shows is its relationships with the external entities.





3.2 Top Level DFD

For a diagram to be called a DFD, it needs to show the inner workings of an information system. The different levels of a DFD indicate how detailed it is. Level 0 (Top Level) DFD is a broad overview of a system, showing hardly any detail within the system – shows the functional components in the software package. **Each component shown in the Top Level DFD is described in the subsections of 4.3.**

A data-flow diagram (DFD) is a way of representing a flow of a data of a process or a system (usually an information system). The DFD also provides information about the outputs and inputs of each entity and the process itself. A data-flow diagram has no control flow; there are no decision rules and no loops. Specific operations based on the data can be represented by a flowchart.

Components

A data flow diagram graphically represents:

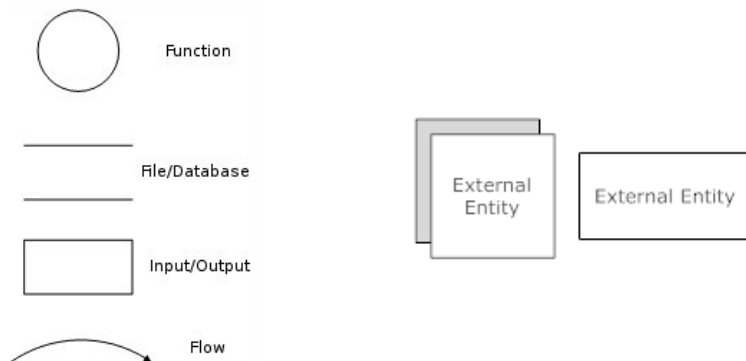
processes - jobs that are done with the data. A process transforms incoming data flow into outgoing data flow.

data stores - files, databases, archives. They can be manual, digital or temporary.

external entities/terminators in a business or other system - other systems or people beyond the control of the current system. These are the places which provide the organisation with data, or have data sent to them by the organisation (e.g. customers, partners, government bodies). External entities are sources and destinations of the system's inputs and outputs.

connecting data flows - arrows show how data flows from one place to another. Flows that cross the system boundary are known as Input Output Descriptions. Label the arrows with the name of the data that moves through it.

Here are the basic DFD shapes :



Rules for creating DFD

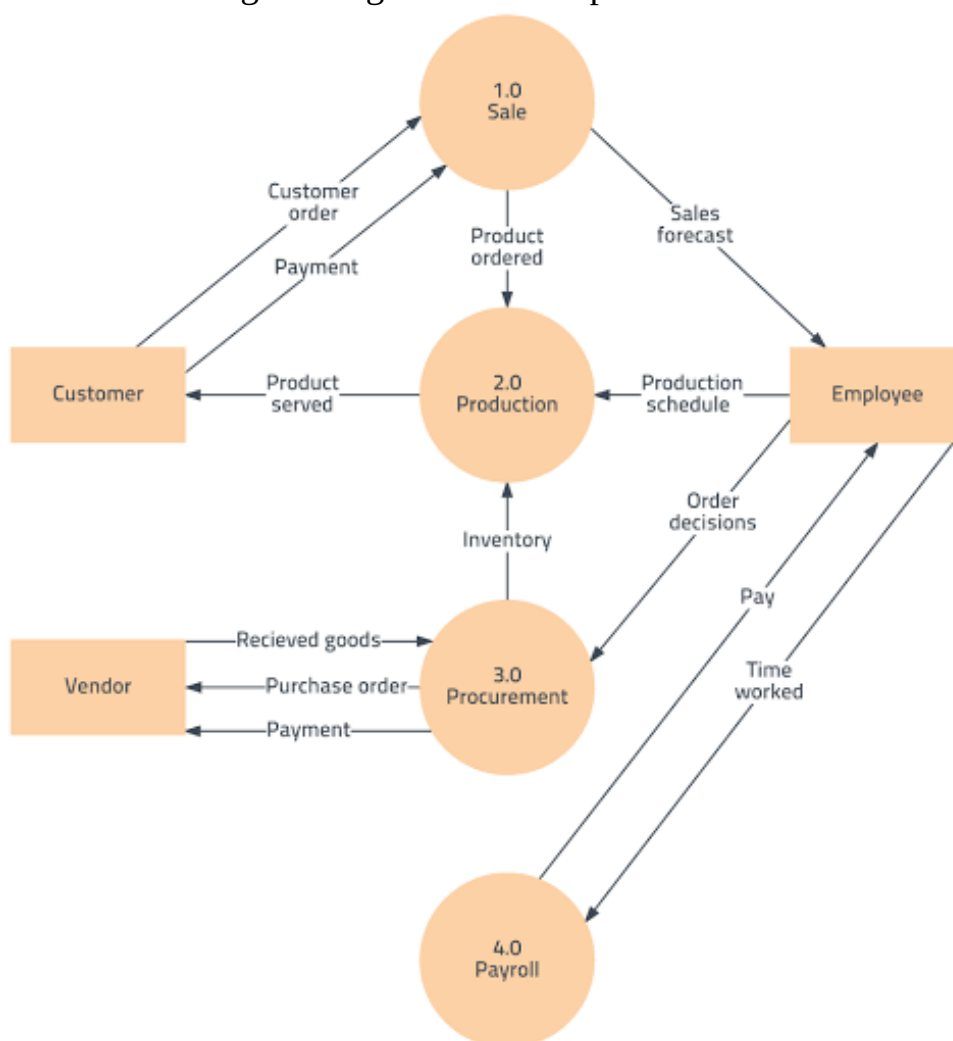
- **Entity names should be comprehensible without further comments.** DFD is a system created by analysts based on interviews with system users. It is determined for system developers, on one hand, project contractor on the other, so the entity names should be adapted for model domain or amateur users or professionals. Entity names should be general (independent, e.g. specific individuals carrying out the activity), but should clearly specify the entity.
- **Processes should be numbered for easier mapping and referral to specific processes.** The numbering is random, however, it is necessary to maintain consistency across all DFD levels (see DFD Hierarchy). DFD should be clear, as the maximum number of processes in one DFD is recommended to be **from 6 to 9, minimum is 3 processes in one DFD.**[1][2] The exception is the so-called contextual diagram where the only process symbolizes the model system and all terminators with which the system communicates.

DFD Conventions

- **Do not allow a single page of a DFD to get too complex - it should have no more than 10 components.** If it has more than this, combine some components into a single self-contained unit and create a new DFD for that unit.
- **Each component and subcomponent should be numbered.** e.g. a top level DFD has components 1 2 3 4 5. The subcomponent DFD of component 3 would have components 3.1, 3.2, 3.3, and 3.4; and the subsubcomponent DFD of component 3.2 would have components 3.2.1, 3.2.2, and 3.2.3. This enables a developer to plan in a top-down manner: starting with representing large concepts, and then repeatedly breaking these objects into their components.
- **All processes must have at least one data flow in and one data flow out.**

All processes should modify the incoming data, producing new forms of outgoing data.

- Each data store must be involved with at least one data flow.
- Each external entity must be involved with at least one data flow.
- A data flow must be attached to at least one process
- **Data flows cannot go directly from one external entity to another external entity:** such flows need to go through at least one process



3.3 Description of components

Provide a technical description of the component. Repeat this section as needed. Items included in the description are dependent upon the type of component (e.g., stored procedure, template, etc.). For all component types, provide the name, type, definition, location, constraints, and logic (either narrative or pseudo code). Include items such as input parameters, output parameters/result sets, and tables/files accessed as appropriate.

5.3. Description of components

5.3.1. Functional component 1

- 5.3.1.1. Input
- 5.3.1.2. Process definition
- 5.3.1.3. Output
- 5.3.1.4. Interfaces with other Functional components
- 5.3.1.5. Resource allocation
- 5.3.1.6. Scope of prototyping (if applicable) - user interface / feasibility study

5.3.2. Functional component 2

- 5.3.2.1. Input
- 5.3.2.2. Process definition
- 5.3.2.3. Output
- 5.3.2.4. Interfaces with other Functional components
- 5.3.2.5. Resource allocation
- 5.3.2.6. Scope of prototyping (if applicable) - user interface / feasibility study

. . .

5.3.n Functional component n

. . .